INSTITUT TECCART 2026

3030 Hochelaga, Montréal, Québec, H1W 1G2

RCS: Laboratoire 1

Objectif

Ce laboratoire a pour objectif de développer une application web full stack (React + Express + MongoDB) permettant de rechercher des films et des recettes via des API externes, de générer des suggestions personnalisées à l'aide d'une intelligence artificielle, et de sauvegarder ces recommandations dans une base de données locale.

À travers ce laboratoire, vous mettrez en pratique les concepts abordés à date :

- La communication entre un frontend React et un backend Express via des appels HTTP.
- L'intégration d'API externes (OMDb, Spoonacular) pour la recherche de données. - L'utilisation d'une API d'intelligence artificielle (OpenRouter) pour la génération de contenu.
- Les opérations CRUD (Create, Read, Update, Delete) avec MongoDB.

Fonctionnement global:

L'application permet à l'utilisateur de rechercher des films ou des recettes, d'obtenir des suggestions personnalisées générées par une intelligence artificielle, puis de sauvegarder ces suggestions pour les consulter plus tard. L'interface propose deux modes accessibles via un sélecteur : le mode Films et le mode Recettes.

Films

L'utilisateur peut rechercher un film par son titre. L'application interroge une source de données externe et affiche une liste de films correspondants. L'utilisateur peut ensuite cocher un ou plusieurs films parmi les résultats, puis remplir ses préférences personnelles : son humeur actuelle, le temps qu'il souhaite consacrer au visionnage, et avec qui il compte regarder le film. En cliquant sur le bouton de génération, l'application envoie les films sélectionnés ainsi que les préférences à un service d'intelligence artificielle, qui retourne une suggestion de film accompagnée d'une explication sur pourquoi ce film convient à la situation décrite.

L'utilisateur peut ensuite sauvegarder cette recommandation. Toutes les recommandations sauvegardées apparaissent dans une section historique, où chaque entrée affiche le titre recherché, les préférences saisies (humeur, durée, compagnie) et la suggestion de l'IA. L'utilisateur peut aussi supprimer une recommandation de l'historique.

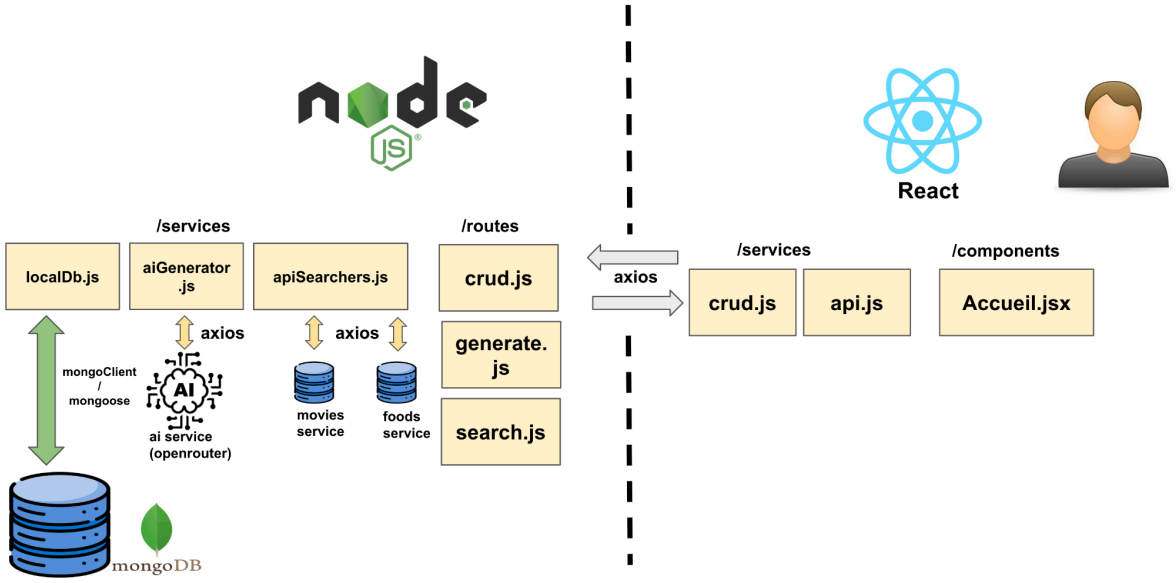
Recettes

L'utilisateur peut rechercher une recette ou un ingrédient par son nom. L'application interroge une source de données externe et affiche une liste de résultats correspondants. L'utilisateur peut cocher un ou plusieurs éléments parmi les résultats, puis préciser ses préférences : son régime alimentaire, le temps dont il dispose pour cuisiner, son budget, son niveau de compétence en cuisine, et pour combien de personnes il cuisine.

En cliquant sur le bouton de génération, l'application envoie les recettes sélectionnées ainsi que les préférences au service d'intelligence artificielle, qui retourne une suggestion de recette accompagnée d'une justification. L'utilisateur peut sauvegarder cette recommandation. L'historique des recettes sauvegardées affiche pour chaque entrée le titre, les préférences (régime, temps, budget, niveau, nombre de personnes) et la suggestion de l'IA. L'utilisateur peut supprimer une recommandation de l'historique.

Travail à faire

Vous devez compléter l'ensemble des fichiers décrits ci-dessous pour que l'application soit fonctionnelle. Le code de base (structure des fichiers, dépendances, configuration du serveur Express et de l'application React) est déjà en place. Votre travail consiste à implémenter les fonctions dans les fichiers des dossiers services/ et routes/ du backend, ainsi que services/ et components/ du frontend.



Backend

Fichier services/apiSearcher.js Ce fichier contient les fonctions qui communiquent avec les API externes pour effectuer les recherches.

- searchMovies(query) : Cette fonction reçoit un terme de recherche en paramètre et interroge l'API OMDb (http://www.omdbapi.com/) pour chercher des films. Elle doit

envoyer une requête GET avec les paramètres s (le terme de recherche), apikey (la clé d'API fournie) et type fixé à movie. Si la réponse contient des résultats (Response === 'True'), la fonction doit retourner un tableau d'objets contenant uniquement le titre de chaque film ({ title: movie.Title }). Si aucun résultat n'est trouvé, elle retourne un tableau vide.

- searchRecipes(query) : Cette fonction reçoit un terme de recherche et interroge l'API Spoonacular (http://api.spoonacular.com/food/ingredients/autocomplete) pour chercher des recettes/ingrédients. Elle doit envoyer une requête GET avec les paramètres query (le terme de recherche) et apiKey (la clé d'API fournie). Elle retourne directement les données de la réponse. En cas d'erreur, elle retourne un tableau vide.

Fichier services/aiGenerator.js Ce fichier contient les fonctions qui communiquent avec l'API d'intelligence artificielle (OpenRouter) pour générer les suggestions.

- queryAIGenerator(query) : Cette fonction reçoit un objet contenant selectedMovies (tableau de titres de films), mood, watchTime et 2 watchWith. Elle envoie une requête POST à l'API OpenRouter (https://openrouter.ai/api/v1/chat/completions) avec le modèle openai/gpt-4o-mini. Le message envoyé au modèle doit inclure la liste des films sélectionnés, l'humeur, le temps de visionnage et la compagnie, en demandant une suggestion d'un film avec une explication. La fonction retourne le contenu textuel de la réponse de l'IA.
- queryAIGeneratorForRecipes(query) : Cette fonction reçoit un objet contenant selectedRecipes (tableau de noms de recettes), diet, timeAvailable, budget, skillLevel et cookingFor. Elle envoie une requête POST à l'API OpenRouter avec les mêmes paramètres de configuration. Le message envoyé au modèle doit inclure les recettes sélectionnées et toutes les préférences culinaires, en demandant une suggestion de recette avec une explication. La fonction retourne le contenu textuel de la réponse de l'IA.

Fichier services/localDb.js Ce fichier contient les fonctions qui interagissent avec la base de données MongoDB locale pour les opérations CRUD sur les recommandations.

- getAllRecommendations(category) : Cette fonction reçoit une catégorie ("movies" ou "recipes") et retourne toutes les recommandations de cette catégorie depuis la collection recommendations de la base de données. Elle utilise find({ category }) suivi de .toArray() pour obtenir un tableau de résultats.
- createRecommendation(data) : Cette fonction reçoit un objet contenant les données d'une recommandation et l'insère dans la collection recommendations avec insertOne(data). Elle retourne ensuite le document complet qui vient d'être inséré en le recherchant par son _id nouvellement généré.
- updateRecommendation(id, data) : Cette fonction reçoit un identifiant et un objet de données à mettre à jour. Elle effectue un updateOne sur le document correspondant à l'identifiant donné, en utilisant l'opérateur $set pour modifier uniquement les champs fournis.
- deleteRecommendation(id) : Cette fonction reçoit un identifiant et supprime le document correspondant de la collection recommendations en utilisant deleteOne avec un filtre sur le _id (converti en ObjectId).

Fichier routes/search.js Ce fichier définit les routes Express pour la recherche via les API externes.

- Route GET /movie : Cette route extrait le paramètre de requête q (le terme de recherche), appelle la fonction searchMovies du service apiSearcher, et retourne les résultats sous la forme { results: results }. En cas d'erreur, elle retourne un statut 500 avec un message d'erreur.
- Route GET /recipe : Cette route extrait le paramètre de requête q, appelle la fonction searchRecipes du service apiSearcher, et retourne les résultats sous la forme { results: results }. En cas d'erreur, elle retourne un statut 500 avec un message d'erreur.

Fichier routes/generate.js Ce fichier définit les routes Express pour la génération de suggestions par l'IA.

- Route GET / : Cette route extrait les paramètres de requête selectedMovies, mood, watchTime et watchWith. Elle doit convertir selectedMovies de chaîne JSON en tableau (avec JSON.parse), puis appeler la fonction queryAIGenerator du service aiGenerator. Elle retourne la réponse sous la forme { response: result }.
- Route GET /recipes : Cette route extrait les paramètres de requête selectedRecipes, diet, timeAvailable, budget, skillLevel et cookingFor. Elle doit convertir selectedRecipes de chaîne JSON en tableau (avec JSON.parse), puis appeler la fonction queryAIGeneratorForRecipes du service aiGenerator. Elle retourne la réponse sous la forme { response: result }.

Fichier routes/crud.js Ce fichier définit les routes Express pour les opérations CRUD sur les recommandations sauvegardées dans MongoDB.

- Route GET /recommendations : Cette route extrait le paramètre de requête category, appelle la fonction getAllRecommendations du service localDb, et retourne les résultats sous la forme { results: results }.
- Route POST /recommendations : Cette route récupère les données du corps de la requête (req.body), appelle la fonction createRecommendation du service localDb, et retourne un message de succès accompagné du document créé sous la forme { message: '...', result: result }.
- Route PUT /recommendations : Cette route extrait id et le reste des données depuis le corps de la requête, appelle la fonction updateRecommendation du service localDb, et retourne un message de succès accompagné du résultat de la mise à jour.
- Route DELETE /recommendations : Cette route extrait le paramètre de requête id, appelle la fonction deleteRecommendation du service localDb, et retourne un message de succès accompagné du résultat de la suppression.

Frontend

Fichier services/api.js Ce fichier contient les fonctions qui communiquent avec le backend pour la recherche et la génération IA.

- searchMovie(title) : Cette fonction envoie une requête GET vers http://localhost:5000/search/movie en passant le titre comme paramètre de requête q. Elle retourne les données de la réponse. En cas d'erreur, elle retourne { results: [] }.
- searchRecipe(name) : Cette fonction envoie une requête GET vers http://localhost:5000/search/recipe en passant le nom comme paramètre de requête q. Elle retourne les données de la réponse. En cas d'erreur, elle retourne { results: [] }.
- generateAIResponse(query) : Cette fonction envoie une requête GET vers http://localhost:5000/generate en passant l'objet query complet comme paramètres de requête. Cet objet contient selectedMovies (en format JSON stringifié), mood, watchTime et watchWith. Elle retourne les données de la réponse.
- generateAIResponseForRecipes(query) : Cette fonction envoie une requête GET vers http://localhost:5000/generate/recipes en passant l'objet query complet comme paramètres de requête. Cet objet contient selectedRecipes (en format JSON stringifié), diet, timeAvailable, budget, skillLevel et cookingFor. Elle retourne les données de la réponse.

Fichier services/crud.js Ce fichier contient les fonctions qui communiquent avec le backend pour les opérations CRUD sur les recommandations.

- showAllRecommendations(category) : Cette fonction envoie une requête GET vers http://localhost:5000/crud/recommendations en passant la catégorie comme paramètre de requête. Elle retourne les données de la réponse. En cas d'erreur, elle retourne { results: [] }.
- insertRecommendation(recommendation) : Cette fonction envoie une requête POST vers http://localhost:5000/crud/recommendations avec l'objet recommendation dans le corps de la requête. Elle retourne les données de la réponse. En cas d'erreur, elle retourne { results: [] }.
- deleteRecommendation(id) : Cette fonction envoie une requête DELETE vers http://localhost:5000/crud/recommendations en passant l'identifiant comme paramètre de requête id. Elle retourne les données de la réponse. En cas d'erreur, elle retourne { results: [] }.
- updateMovie(id, movie) : Cette fonction envoie une requête PUT vers http://localhost:5000/crud/recommendations avec l'identifiant et les données du film dans le corps de la requête. Elle retourne les données de la réponse. En cas d'erreur, elle retourne { results: [] }.

Fichier components/Accueil.jsx Ce fichier contient le composant principal de l'application. Il gère l'ensemble de l'interface utilisateur et la logique d'interaction.

Organisation du travail

le travail doit se faire en binôme, ou le premier membre s’occupe des fonctionnalités des films, et le deuxième travaille sur les fonctionnalités des recettes

Critères d’évaluation

Element	Pondération

Livrable attendu

Rapport des prompts utilisé dans github copilot + le lien github du dépôt ou vous avez collaborer.